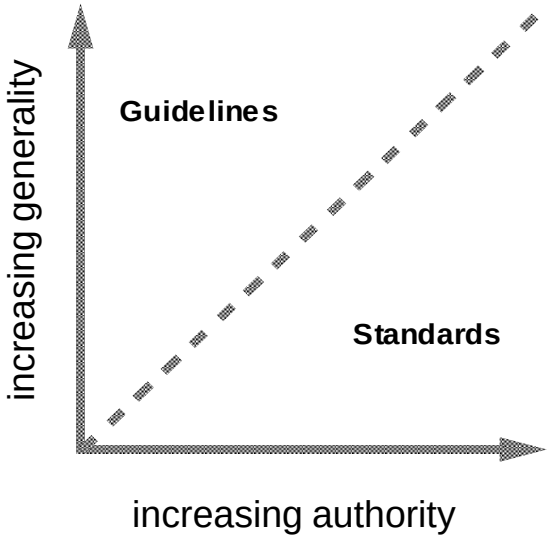
DESIGN RULES

Design rules

- What is the goal of interaction design?
  - Designing for the maximum **usability**
- Types of design rules
  - Standards/Guidelines/Principles
- Standards and guidelines
  - Direction for design

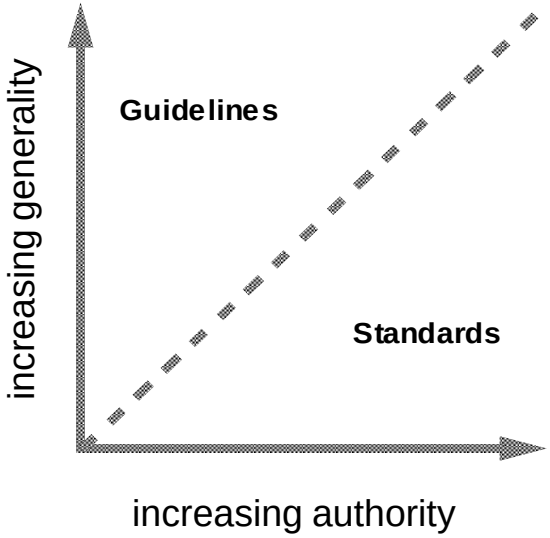
Types of design rules

- We can classify these rules along two dimensions based on the rule's authority and generality
- By authority, we mean
  - Rule must be followed in the design or
  - It is only suggested
- By generality, we mean
  - Rule can be applied to many situations/applications or
  - It is focused on a limited situations/application



Types of design rules (contd.)

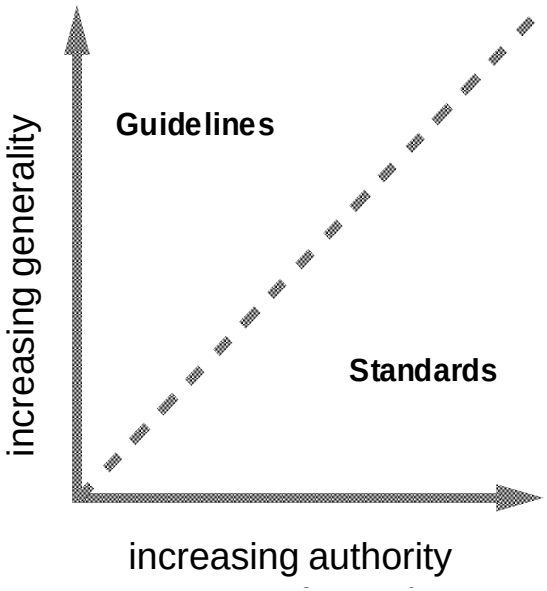
- Standards
  - Specific design rules
  - High authority
  - Low generality (Limited application)
- Guidelines
  - Lower authority
  - High generality (More general application)
- Principles
  - Abstract design rules
  - Lower authority
  - High generality



Using design rules

Design rules

- Suggest how to increase usability
- Differ in generality and authority



Standards

- Higher level of authority but so useful for specific design
- Set by national or international bodies to ensure compliance by a large community of designers standards require sound underlying theory and slowly changing technology
- Hardware standards more common than software high authority and low level of detail

Guidelines

- More technology oriented, but they are also general
- Abstract guidelines applicable during early life cycle activities
- Detailed guidelines (style guides) applicable during later life cycle activities
- ISO 9241 defines usability as effectiveness, efficiency and satisfaction with which users accomplish tasks

- Understanding justification for guidelines aids in resolving conflicts

Principles

- Principles are derived from knowledge of the psychological, computational and sociological aspects of the problem domain
- Independent of the technology
- Therefore can be applied to widely but not so useful for specific design

Golden rules and heuristics

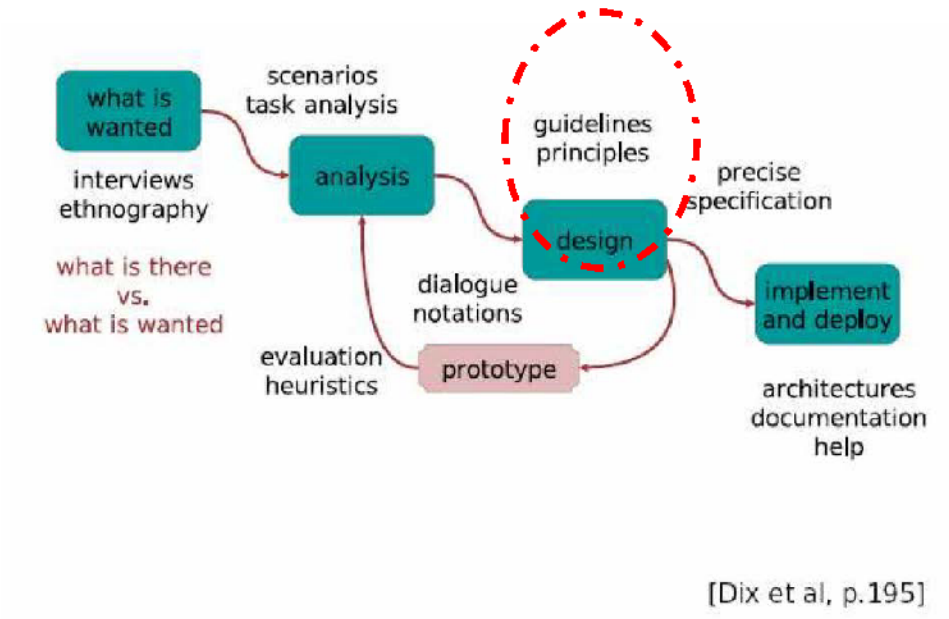
- "Broad Brush" design rules
- Useful check list for good design
- Different collections of design rules includes such as
  - Shneiderman's 8 Golden Rules
  - Norman's 7 Principles
  - Nielsen's 10 Heuristics

Background

We use Dix et al's version of the Waterfall model (reproduced below from Dix et al) to illustrate where exactly in the design cycle, the Rules - Guidelines & principles being discussed in this module, become important.

HCI in the design process

- Waterfall model



[Dix et al, p.195]

Design is both qualitative as well as quantitative.

The terms 'Guidelines' and 'principles' rather than precise' laws & rules' are used in Design.

Ben Shneiderman formulated eight such guidelines that can be used in Interface designing.

Introduction

Ben Shniderman* consolidated known tacit knowledge and practice guidelines which are used intuitively by graphic Interface designers - into a set of eight general guidelines for the use of computer science specialist who were being introduced to Visual Graphic designers' work of designing interactive GUI interfaces. Along with 'looks' the usability of a software depended on functionality.

**GUI:Graphic User Interface

* Bend Shneiderman founded the HCI Lab at the University of Maryland , USA. He is known for Nassi-Shnerderman diagrams used in the field of Software Engineering.

- There is ample empirical evidence published in HCI literature which collaborates and consolidates the applicability of the eight guide lines.
- These are intended more as guidelines rather than ‘rules’ to be strictly adhered to at every step.
- They are useful for designers as well as software engineers involved in design of interfaces.
- Using these eight guidelines it is possible to distinguish a good interface design from a bad one especially from the Human - User interaction point of view.
- These have been put forth in a concise and understandable manner by Ben Shneiderman.

- It needs to be noted that apart from these eight there are many more similar useful pointers available in HCI and Usability literature.

- While merely or blindly applying these eight guidelines is not necessarily going to grantee a good interface ‘design’, they are useful in heuristic evaluation to identify GUIs that fall out of normal ‘pattern’ . The guidelines can be used to rate GUI’s as good or bad.

SHNEIDERMAN'S 8 GOLDEN RULES

Shneiderman's 8 Golden Rules

1. *Strive for consistency*
2. *Cater to Universal Usability (Enable frequent users to use shortcuts)*
3. *Offer informative feedback*
4. *Design dialogs to yield closure*
5. *Offer error prevention and simple error handling*
6. *Permit easy reversal of actions*
7. *Support internal locus of control*
8. *Reduce short-term memory load*

Explanations & Examples

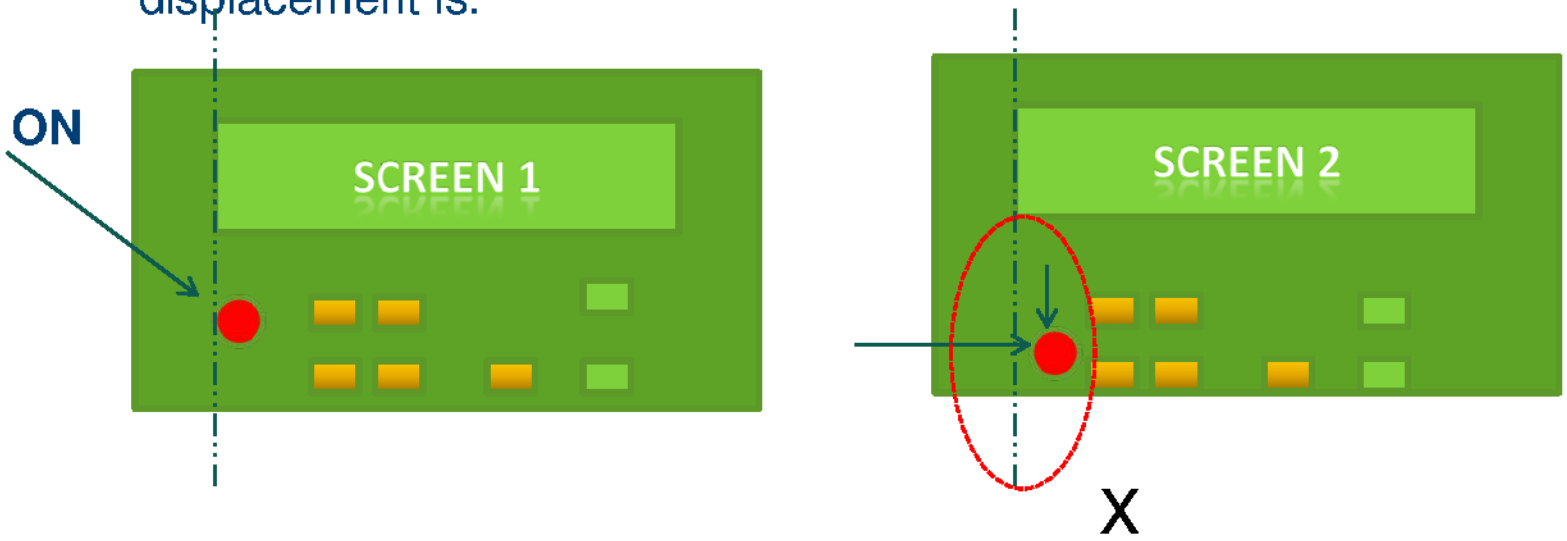
1. Strive for Consistency

- Users need to be able to do the same thing the same way that they have been doing.- every time.
- Interfaces need to exhibit 'consistent' quality across screens/ applications both visually as well as behaviorally.
- Consistency leads to a pattern which is easier to handle cognitively.
- Consistency such as 'similar sequence of actions in similar situations' makes it easy to learn.

 Continued.....

Consistency can be achieved through graphical elements such as fonts, colour, shape, position being consistently same in all menus & screens, across, categories for a particular software.

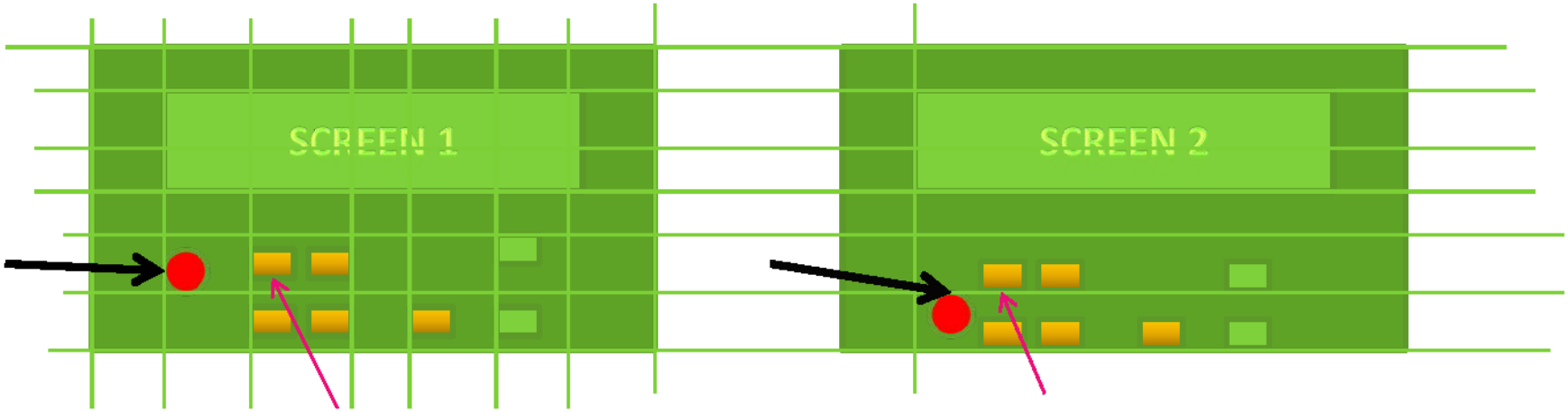
For example: If the **ON** button is on the right in the first screen and moves towards middle in the second screen then positional inconsistency is said to have occurred - however small the displacement is.



GUI designers use a simple technique to maintain consistency of control elements in successive screen.

Consistency….. continued

GUI designers use a background grid to place interactive elements in a consistent and orderly way so as to make them appear both physically as well a visually at the same place across the entire software package.



Inconsistent positioning of GUI elements is evident when observed against a grid. Grids are used as background reference to place the elements consistently

In case certain exceptions in maintaining consistency are required to be made in a subsequent screen, they should be such that they are comprehensible, distinct and limited in number.

2. Cater to wide range & type of Users

1. Strive for Consistency
2. **Cater to Universal Usability**
3. Offer Informative feedback
4. Design Dialogs to yield closure
5. Prevent Errors
6. Permit easy reversal of actions
7. Support internal locus of control
8. Reduce short term memory load

Universal design strives to cater to as wide a range of human users of different characteristics (age, culture, educational level, disability) with a single design.

While this may not be feasible or possible in all contexts, Shniderman's rule none the less needs to be followed so as not to leave out taking into consideration a section of users, other wise competent, who cannot use the interface due to no fault of theirs.

Users are classified as **Novice**, **Intermediate** and **Experts**. Experts tend to use lesser actions at a faster pace. Abbreviations short cuts keys etc are some of the techniques used.

Interfaces need to cater to all levels & classification of users: novice to experts.

3. Offer Informative feedback

1. Strive for Consistency
2. Cater to Universal Usability
3. **Offer Informative feedback**
4. Design Dialogs to yield closure
5. Prevent Errors
6. Permit easy reversal of actions
7. Support internal locus of control
8. Reduce short term memory load

- Interfaces need to not just to be communicative but also need to inform the 'user' in terms of learning & feed back which tells them that they are proceeding in the right direction.
- For every action of the user there needs to be a feedback – only then 'interaction' (in HCI) is said to take place. Specific error messages composed in a appositive tone give affirmative feedback without having to feel punitive.
- Unless the user gets a feed back he/she cannot proceed or becomes unsure of the correctness of the action.

4. Design Dialogs to yield closure

- In an interaction - dialogue needs to have a closure which is recognized by the user as end of an action.
- Sequence of actions need to proceed in a dialogue by engaging the user in a step by step manner.
- Like in a mathematical expression, every enclosing bracket needs a corresponding closing bracket. So also subsequence of actions needs to be grouped with intermittent closing of each sub group followed finally by a closer action of the group.

Ex: A message at the end of a sequence of events gives a feed back & closure of sending a SMS.

Continued.....

5. Prevent Errors

1. Strive for Consistency
2. Cater to Universal Usability
3. Offer Informative feedback
4. Design Dialogs to yield closure
5. Prevent Errors
6. Permit easy reversal of actions
7. Support internal locus of control
8. Reduce short term memory load

Interfaces need to minimize errors. Human Computer dialogue can be designed to minimize and prevent errors made by users.

There cloud be many reasons for users errors but the user himself or herself is not one of them ! Users can make errors while interacting with computers as well as while inputting / interpreting information.

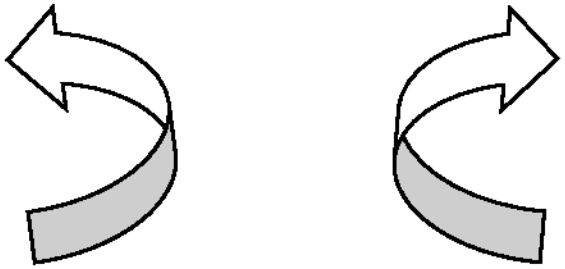
Even if the user makes an error the system needs top be designed to detect it, take corrective or precautionary steps to arrest it. It also needs to offer a way out for recovery from the error.

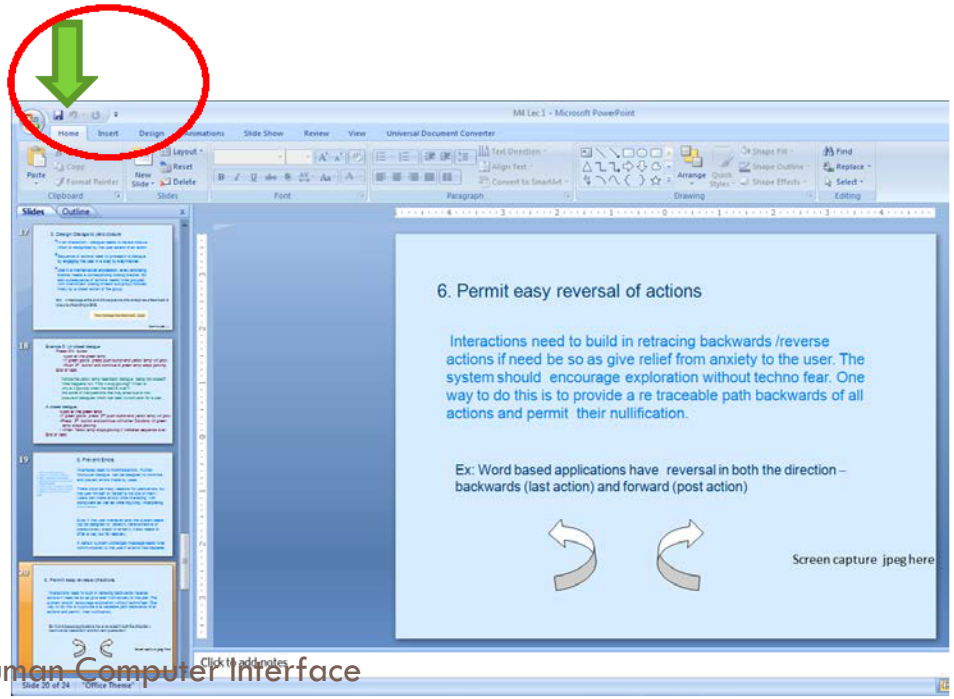
A default system unchanged message needs to be communicated to the user if an error has happened .

6. Permit easy reversal of actions

Interactions need to build in retracing backwards /reverse actions if need be so as give relief from anxiety to the user. The system should encourage exploration without techno fear. One way to do this is to provide a re traceable path backwards of all actions and permit their nullification.

Ex: This PPT application has reversal in both the direction – backwards (last action) and forward (post action)



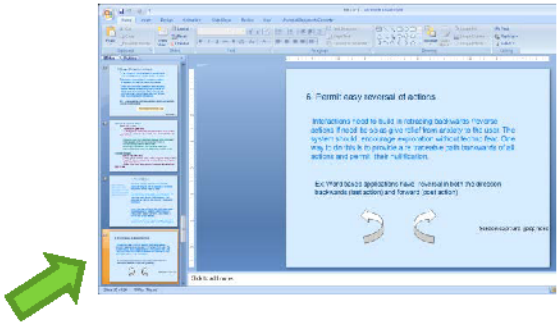


7. Support internal locus of control

Allow user to always feel ‘in control’ of the system and of the situation.

Make the user aware that he/she is in control. User should believe that they are controlling the system and not the other way around. This is achieved by more opportunities for ‘interactions’.

The bearing of where the user presently is helps the user to orient or reorient the interaction. The user should never be allowed to feel lost.



8. Reduce short term memory load

1. Strive for Consistency
2. Cater to Universal Usability
3. Offer Informative feedback
4. Design Dialogs to yield closure
5. Prevent Errors
6. Permit easy reversal of actions
7. Support internal locus of control
8. **Reduce short term memory load**

94 56 781029

Easier to remember if chunked into smaller setc

94 56 7 810 29

Care not load the cognitive short term memory of the user by expecting user to remember several sequences , actions and their consequences at a time. Means loading their short term memory while interacting.

Millers* 7 chunks of information is often prescribed as a solution to limit short term memory. In psychological experiments it has been found that the short term memory can hold 7 +- 2 bits called chunks of information. Long sequential actins requiring more than 7 chunks need to be broken down into smaller chunks.

*G.A. Miller; The Magical number seven, plus or minus two: some limits on our capacity to process information. Psychplogical review, 63(2):81-97, 1956.

Each of these Shneiderman's rules were examined with the examples

1. **Strive for Consistency**
2. **Cater to Universal Usability**
3. **Offer Informative feedback**
4. **Design Dialogs to yield closure**
5. **Prevent Errors**
6. **Permit easy reversal of actions**
7. **Support internal locus of control**
8. **Reduce short term memory load**

NORMAN'S 7 PRINCIPLE

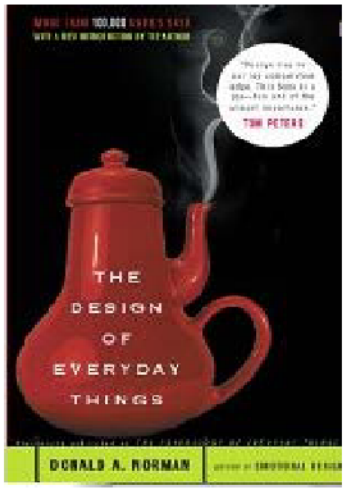
Norman's 7 Principles

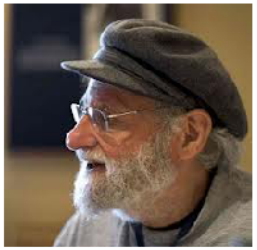
1. *Use both knowledge in the world and knowledge in the head.*
2. *Simplify the structure of tasks.*
3. *Make things visible: bridge the gulfs of execution and evaluation.*
4. *Get the mappings right.*
5. *Exploit the power of constraints, both natural and artificial.*
6. *Design for error.*
7. *When all else fails, standardize.*

Introduction:

In 1988 Donald Norman proposed **seven** principles for the evaluation of the interaction between Humans and Computers.

Donald Norman is a Researcher, Psychologists & Designer well known for his book titled: The Psychology of Everyday things.





Later he formulated a **model** to understand & integrate a user into the Interface design cycle.

He intended seven stages to be used to transform difficult tasks for which HCI & interface was under development into simple ones.

Norman outlined the underlying principles for his 7 stage models as shown bellow. The seven stage Interaction model is shown in subsequent slides.

Principles underlying the seven stage model

1. Use both knowledge in world & knowledge in the head
2. Simplify task structures.
3. Make things visible
4. Get the mapping right

   (User mental model = Conceptual model = Designed model)
5. Convert constrains into advantages

   (Physical constraints, Cultural constraints, Technological constraints)
6. Design for Error
7. When all else fails – Standardize.

Each of these seven principles is discussed in the following slides.

1. Use both knowledge in the world & knowledge in the head

•As a basis for his Interaction Model Norman proposed the following levels of abstraction of knowledge of the user :

- Task Level
- Goal Level
- Semantic level
- Syntax level
- Lexical level
- Physical Level

The User models his/her knowledge in/of the world into his / mental realm by the process of cognation.
The user's knowledge model is not necessarily the same as the knowledge model of the world.

The question is which one should the designer take as reference –
Knowledge in the World ?
or
Model of world knowledge in the User's Mental realm ?

Continued....

Relying on either of them alone would lead to an incomplete abstraction of knowledge by the Designer.
Hence Norma's principle mandates that both types of knowledge be considered.

- **Semantic level** describes the set of objects, attributes and operations, which the 'system' and the 'user' can communicate. Semantics is about how the user interprets and makes meanings out of the system.

- **Syntactic level** describes which conceptual entities and operations may be referred to in a particular command context or system state.

- **Interaction level** describes the translation of commands and objects into the associated physical actions and the structure of the interaction, including typing / mouse / gesture / voice / tactile rules.

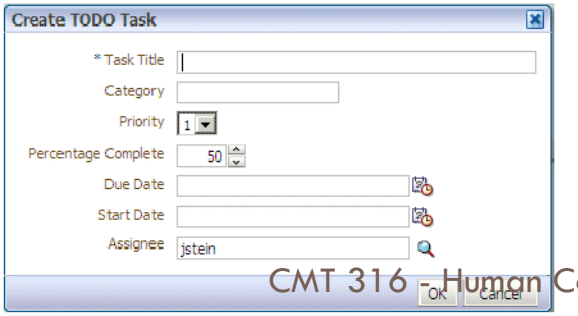
2. Simplify task structures.

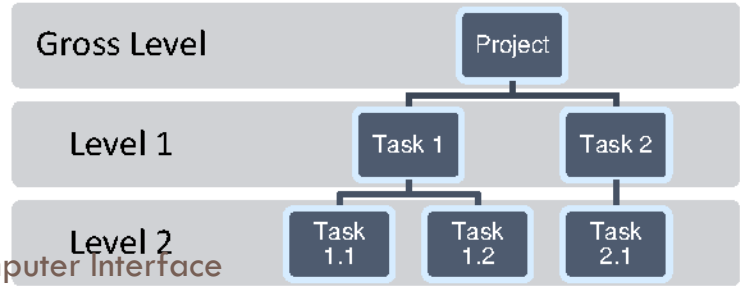
- **Task Level:** task level is to analyze the user's needs and to structure the task domain in such a way, that a computer system can play its part. The task level describes the structure of the tasks which can be delegated to the computer system.

- This principle states that a 'Task' is to be broken down (by analysis) to their simplest action level such that at each level there is as far as possible only one action involved.

- Doing so makes mapping with the Computer's programming langue easy for a HCI designer to build Interactive Interfaces & Hierarchies.

Example: Gross level & Broken Down level of a GUI





3. Make things visible

Objects at the Semantic level need to be **mapped** to the objects at Syntactic level, for the user. This is achieved with making the connection as 'Visual' as possible.

Graphic User Interface designers use ' Metaphors' to make the connection.
Example:

Delete action (at the Semantic Level) (Command Line)

= Waste paper basket to dump. (Syntactic level object) (natural language)

= Visual :

Mapping: the link between what *you want to do* and what *is perceived possible.*

Continued...

Discussions on Normans seven principles. 3 of 7. Continued from previous slide….

HCI Designers use this principle of 'Making it Visual ' to the maximum while designing Interfaces.

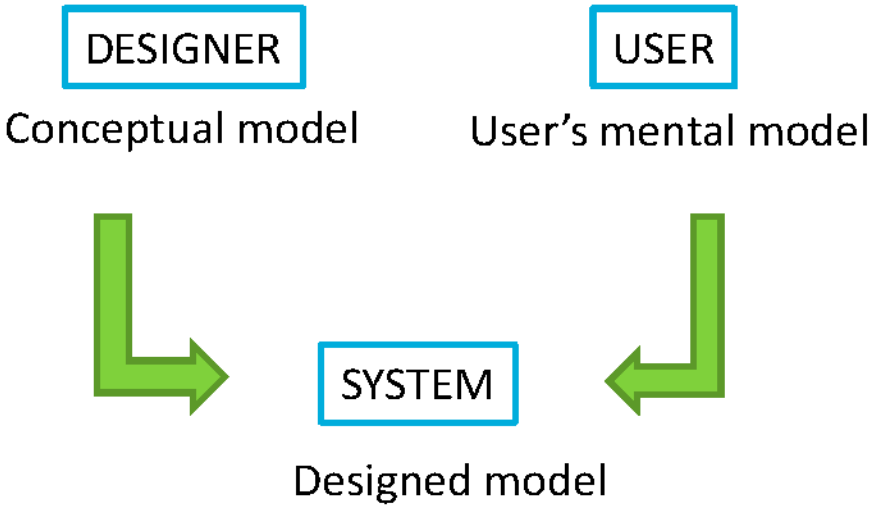
Interaction styles such as
WIMP (Windows – Icons – Menus – Pointer)
Three Dimensional Interfaces as in
Virtual Realty can be found in current software interfaces.

Example of 'Visual':
Windows 8 Interface.

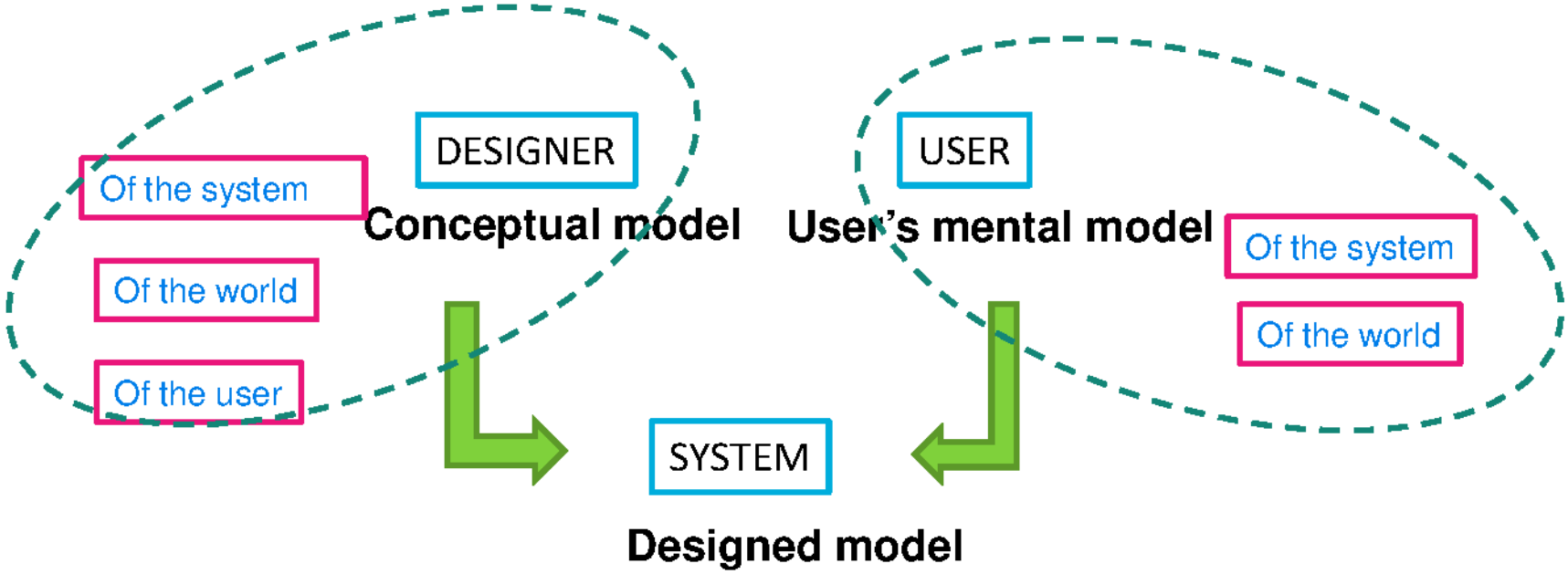
4. Get the mapping right

Mapping: the link between what *user wants to do* and what *is perceived as possible - by the user based on the user's own logic.*

User Mental Model = Conceptual Model = Designed Model



The three models are elaborated upon in the next slide.

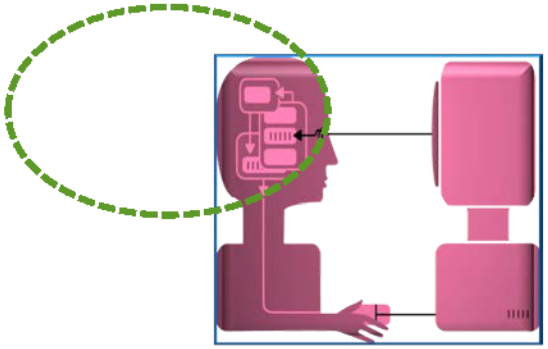


The **User's Mental Model** is the model of a system's working that a user creates when learning and using a computer. It is not technically accurate. It may also be not stable over time. User's mental models keep changing & evolving as learning continues.

In a way Mental Models are models people have of themselves, others and environment. It is their inner understanding.

The mental model of a device is formed by interpreting its perceived actions and is a visible physical structure. Some times the word 'System image model' is also used to imply the real world physical model.

The Conceptual Model.

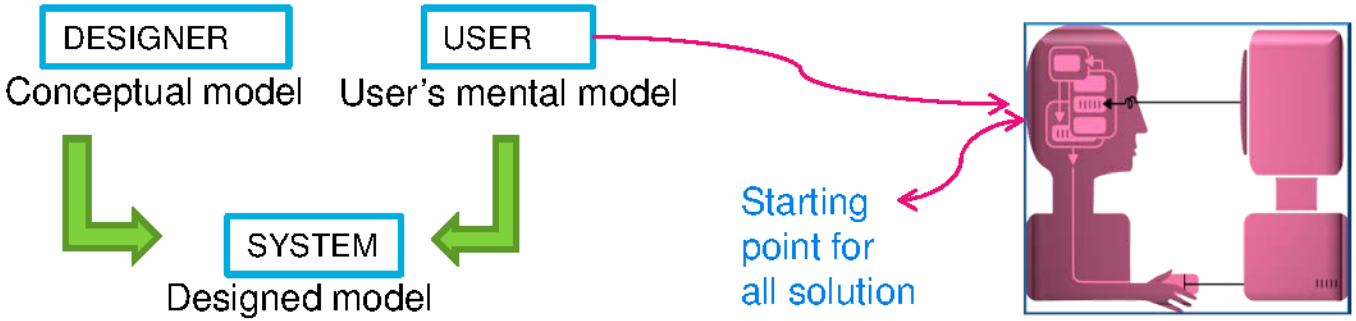


- This is a technically accurate model of the computer / device / system created by designers / teachers/researchers for their specific internal technical use.

- Users too have a Conceptual model but it is their mental model unless the user is as technically qualified as the evaluator. In a way as far as the user is concerned mental models and conceptual models are inherent to each other.

- Designer's too have Mental models of the system. So a Conceptual model of the system needs to be as close as possible to the System's Image Model.

Continued.....



A good device / system will emerge when the starting point of the design process is the user- his/her mental model' (in turn derived through user research- task analysis, walk –through, Contextual inquiry etc) being the basis of the system image and its conceptual model.

The Conceptualised solution the Designer had in his/her mind is called the *design model* .

The User model (what the user develops in the self to explain the operation of the system) and the system image (the system's appearance, operation way it responds is usually a blend of the users mental model and conceptual model all rolled into one. (unless the user happens to be an expert)

Ideally, the design model and user model have to be as close as possible for the systems acceptance. The designer must ensure that the system image is consistent with and operates according to the proper conceptual model.

5. Convert constrains into advantages

This is about how to ensure that the user knows what to do next when there are more than one possibility or more than one given option.

In other words how a designer needs to embed constraints in the sequence of operations in an interface such that the user is guided to the right sequence choice by reducing the chance of error in choosing the wrong option

As a principle Interfaces need to have any of the three type of constraints Physical; Technological; Cultural

Physical constraints :
Based on shape , size, area (example mouse over area demarcation)

Continued....

Cultural constraints : Culturally & semantically practiced rituals, symbols, color codes.

Ex: Always start from Right & stop on Left lower end in a word document.

Technological constraints: Example: Closing a file without saving – user needs to be warned every time this is likely to be operated by the user.

Program it such that it is mandatory to press the save button before close button.

Visibility and feedback:
Visual design also can suggest constraints.
Ex: If a number of identical buttons are required for diverse functions, Visually building differences such as colour or grouping , it is possible to ‘ constrain’ a user in not pressing randomly the identical looking buttons placed in close proximity.

6. Design for Error

Errors are not taken as human faults in users in HCI.
This means Errors by users cannot be blamed on Users.
Users are not the cause for errors.
Often errors are '**slips**' - intend to do one thing but end up doing another accidentally.
Errors happen when there is a mismatch between User's mental model, designers' understanding of User's mental model; system limitations.

Research literature revels that Errors can be classified as:

Description Errors: Two objects physically alike are described / taken mistakenly for each other.
One solution employed is ' highlighting' the object which is in line of next action so that ' attention' is drawn to that right object from amongst similar looking group of objects.

Data Errors: Could be perception errors or selection errors. A solution could be reversal of action without penalty and ' affordance' by the user to correct the error by retracing action steps.

Associative Action Errors:

Associative Errors are those that involve activating one sequence in place of another and realising it when the wrong /unexpected response results.

Associative Errors also happen when short term memory is overloaded or long term memory fails. Forgetting to do something as prescribed or reversing the sequence - Pressing the second button first instead of the first button etc

- ‘Slips’ & Errors need to be taken care of in Design by providing feed back (either pre or post action). Example : Prompting.

- The cause of the error needs to be understood more than the error.

- Retracing actions must be provided for.

- Assume Task to be imperfect and assume that users will always approximate their actions.

7. When all else is unsuitable – Standardize.

In certain situations / contexts wherein the nature of the task is critical , the user needs to be ‘ forced’ to follow a the only choice as given (afforded) by design.
Example of such situations are : Medical Devices; Warfare equipment; Nuclear Equipment; Power Plant Controls, Energy Grids; Air Traffic Controls etc

In such critical application contexts ‘ STANDARDISATION’ practice is followed.

However very stringent usability testing & evaluation practice is followed before ‘**standardising**’ the format for both INPUT as well as the OUTPUT .

Standardisation comes under the ‘ Best Practice’ adaptation wherein specific rules are the basis;

Where as PRINCIPLES are abstract design rules with navigation and UCD focus , ‘ GUIDELINES’ allow more freedom to the designer.

Between ‘ Standardisation, Principles and Guidelines, the context and the level of the user (expertise) are the determining factors.